

PA 1: Implementing xv6 User Commands and System Calls

```

rekonq@ubuntu-machine:~/xv6-pa1$ make qemu
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/rekonq/xv6-pa1- -march=rv64gc -std=gnu99 -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -no-builtin-strncpy -fno-builtin-strncpy -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-;
no-builtin-free -fno-builtin-memcpy -fno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -fno-builtin-stack-protector -fno-pie -no-pie -c -o user/uptime.o user/uptime.c
riscv64-unknown-elf-ld -z max-page-size=4096 -t user/user.ld -o user/uptime.user/uptime.o user/ulib.o user/usys.o user/print.o user/unalloc.o
riscv64-unknown-elf-objdump -S user/uptime -o user/uptime.asm
riscv64-unknown-elf-objdump -t user/uptime | sed 's/,SYMBOL TABLE/d; s/, * / /; /S/d' > user/uptime.sym
mf/mf %s; ln -nsfme user/cat user/echo user/forktest user/grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_userstests user/_grind user/_wc user/_zombie user/_logstress user/_f
rphan user/_dorphan user/_sync user/_uptime
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1000 blocks have been allocated
ballocc: write bitmap block at sector 40
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -n 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,index=0 -device virtio-blk-device,drive=fs,bus=virtio-mmio-b
0
xv6 kernel is booting

_init: starting sh
$ uptime
 0 11:00:00 up 91 clock ticks
$ uptime
 0 11:00:00 up 91 clock ticks
exec uptime failed
$ uptime
 0 11:00:00 up 182 clock ticks
$

```

```
xv6 kernel is booting

init: starting sh
$ time1
usage: time1 command [args...]
$ time1 echo hello
hello
elapsed time: 0 ticks
$ time1 missingcommand
time1: exec missingcommand failed
elapsed time: 0 ticks
$ time1 sleep 10
elapsed time: 10 ticks
$
```

```
xv6 kernel is booting

init: starting sh
$ time1 matmul
Time: 4 ticks
elapsed time: 5 ticks
$
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

kernel/proc.h: Added uint cputime to struct proc to store the number of CPU ticks

kernel/proc.c: Initialized cputime to zero in allocproc()

kernel/trap.c: Incremented cputime on timer interrupts in both usertrap() and kerneltrap(), accounting for execution in user and kernel mode.

```
xv6 kernel is booting

init: starting sh
$ time1 matmul
Time: 4 ticks
elapsed time: 5 ticks
$
```

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

```
rekonz@UbuntuMachine:~/xv6-pa1$ make qemu
qemu-system-riscv64 -machine virt -bios none
.0

xv6 kernel is booting

init: starting sh
$ wait2tet
exec wait2tet failed
$ wait2test
expected pid=5, returned pid=5
status=7, cpu ticks=10
$
```

For Task 4 the following files were changed:

kernel/pstat.h: Defined struct rusage to hold the child's CPU usage.

user/user.h: Declared struct rusage and the user-callable wait2() function.

user/usys.pl: Added the entry that generates the wait2() system-call stub.

kernel/syscall.h: Assigned syscall number 23 to wait2().

kernel/syscall.c: Registered sys_wait2() in the syscall dispatch table.

kernel/sysproc.c: Implemented sys_wait2() to retrieve the two user addresses and call kwait2().

kernel/defs.h: Declared kwait2() for use by other kernel files.

kernel/proc.c: Implemented kwait2() to wait for a child, copy its exit status and CPU usage into the parent's memory, and then free the child's resources.

user/wait2test.c: Added a test verifying the returned PID, exit status, and CPU ticks.

Makefile: Added wait2test to UPROGS.

The test returned the expected PID, exit status 7, and 10 CPU ticks, confirming that the new syscall worked.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

implemented time to display elapsed time, CPU time, and CPU percentage

```
init: starting sh
$ time
usage: time command [args...]
$ time echo hello
hello
elapsed time: 0 ticks, cpu time: 0 ticks, N/A% CPU
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul
Time: 5 ticks
elapsed time: 6 ticks, cpu time: 6 ticks, 100% CPU
$
```

Two processes running

```
$ time matmul 20 & time matmul 20 &
$ Time: 86 ticks
elapsed time: 87 ticks, cpu time: 43 ticks, 49% CPU
Time: 94 ticks
elapsed time: 94 ticks, cpu time: 51 ticks, 54% CPU
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

I learned how xv6 user programs request services from the kernel through the system calls. I also learned the difference between elapsed time and CPU time. During implementation, I encountered errors when compiling because of incorrect syntax or invalid calls. I resolved these by checking the compiler messages and reviewing how each file connects to the system-call path.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

Our time command only counts CPU time for the process it starts, so work done by that process's children is not included. It also measures whole ticks, so very short programs may show zero time. Results can vary when other processes compete for the CPU.

AI disclosure:

I used AI for implementation guidance, explanations of xv6 system calls and locking, and for troubleshooting errors when compiling.